

Bitácora de Prompts – Proyecto Lúcida

Este documento registra el proceso de desarrollo de la aplicación Lúcida mediante el uso de herramientas de Inteligencia Artificial Generativa, evidenciando la estrategia de Prompt Engineering empleada a lo largo de las dos fases del proyecto (descubrimiento/prototipado y desarrollo).

Para facilitar la lectura, el contenido está organizado mediante un código de colores que distingue el origen de cada intervención:

Amarillo: corresponde a las consultas del grupo de trabajo, instrucciones y prompts, es decir, lo que se solicitó en cada etapa del proceso.

Verde: corresponde a las respuestas, recomendaciones y prompts elaborados por el asistente de IA (Claude), utilizado como apoyo para estructurar la solución, redactar prompts y resolver problemas.

Celeste: corresponde a las respuestas entregadas por Google AI Studio, herramienta con la que se generó y depuró el código de la aplicación funcional.

A través de esta bitácora se busca evidenciar no solo los prompts finales más efectivos, sino también el proceso iterativo de corrección de errores, refinamiento de instrucciones e implementación de la lógica de negocio, conforme a lo solicitado en los requerimientos del proyecto.

Te voy a adjuntar un caso, el cual tengo que resolver para la asignatura de sistema de información. El caso es sobre una persona llamada Lucia y su problemática sobre cómo gestiona su trabajo. Quiero que leas el caso, lo analices y lo comprendas, y posteriormente ocupes el segundo documento adjunto, el cual es un PDF en donde se explican cuáles son los pasos o cuáles son los requerimientos que son necesarios de entregar. También te adjunto tres talleres que se realizaron durante el semestre y con los cuales, básicamente, se pretende que se pueda realizar este caso. Quiero que primero lo analices todo, que analices qué es lo que tienes, lo entiendas, y posteriormente vamos a seguir hablando, pero no quiero que me entreguen nada ahora mismo.

He revisado todo el material. Déjame resumir lo que tengo claro para confirmar que estamos alineados antes de avanzar.

El caso: "Freelance en Apuros" — Lucía

Lucía es diseñadora gráfica independiente con seis clientes fijos y varios esporádicos. Su problema central es el desorden en la gestión: cobra de tres formas distintas (por proyecto, por hora, por suscripción mensual), factura desde Word, registra sus horas en las notas del celular, y lleva los pagos en un Excel desactualizado hace dos meses. La consecuencia concreta: no pudo demostrar que un cliente le debía desde enero.

La misión del grupo es construir un sistema que centralice clientes, proyectos, horas y pagos, con dos componentes: un dashboard financiero personal y una app sencilla con cronómetro para registrar el tiempo trabajado por cliente.

El dashboard debe responder tres preguntas: cuánto ha facturado y cuánto le deben, qué cliente le genera más ingresos y cuál le consume más horas, y cuál es su tarifa efectiva por hora real.

Lo que pide el proyecto (PDF)

Es un proyecto de consultoría tecnológica asistida por IA en dos fases. Fase 1 es descubrimiento y prototipado: analizar requerimientos con NotebookLM y prototipar la interfaz con Stitch. Fase 2 es desarrollo: definir y justificar un stack, generar la base de datos SQL (DDL y DML), backend, frontend, y documentar la estrategia de prompting.

Los entregables van en un repositorio con carpetas /docs (informe ejecutivo, link a Stitch, bitácora de prompts), /db, /backend, /frontend, más un video pitch de máximo 5 minutos. La entrega es por Campus Virtual.

Los talleres

Los tres talleres son la guía metodológica con el caso del restaurante "Appetit" como ejemplo trabajado de punta a punta: cómo usar NotebookLM con Deep Research para investigar y generar un informe, y cómo usar Stitch para prototipar a partir de ese informe, iterar sobre el diseño y crear prototipos navegables. Es el mismo flujo que tendrás que replicar para Lucía.

Una cosa: el Taller 02 y el Taller 03 son archivos .pptx que aún no he abierto en detalle (solo vi el Taller 01 completo). Si quieres que su contenido específico influya en lo que hagamos, dime y los reviso a fondo.

Quiero que también analices los talleres dos y tres, y me comentes.

Taller 02 — "Construyendo una app" (Google AI Studio)

Es el puente entre el prototipo y el código real. El flujo es: tomas la idea ya prototipada en Stitch y la llevas a una aplicación web funcional usando Google AI Studio, que genera el código a partir de un prompt. El ejemplo trabajado es un analizador de inventario que recibe un CSV y devuelve una clasificación ABC por Pareto, lo cual te muestra el patrón a seguir: describes la app, defines qué datos entran, qué procesamiento ocurre y qué dashboard sale.

Lo importante del taller es la cadena técnica que enseña: generas la app en AI Studio, puedes iterarla con más prompts (en el ejemplo agregan filtrado y ordenamiento), descargas el código como .zip, y luego lo ejecutas en StackBlitz (un IDE online) para no tener que instalar nada localmente. Ahí instalas dependencias con npm install y conectas la API Key de Gemini mediante un archivo .env.local. La diapositiva final recuerda que puedes incorporar al prompt los recursos visuales que generaste en Stitch.

Taller 03 — "Desplegando nuestra App" (servicios en la nube)

Es la fase de publicación. Enseña a hacer el deploy para que la app sea visible públicamente. El flujo: conectas AI Studio con GitHub (control de versiones), guardas tu proyecto en un repositorio con commits, y luego despliegas usando Render, enlazando el repositorio de GitHub. Se trabaja con servicios gratuitos (con límites), se configura el plan Free, y se menciona que las variables de entorno y API keys van configuradas en el servicio de despliegue. El resultado es una URL del estilo tuapp.onrender.com.

Lo que esto significa para el caso de Lucía

Ahora tengo el cuadro completo. Los tres talleres juntos trazan exactamente el camino que el proyecto exige:

NotebookLM (Taller 01) para investigar y generar el informe de requerimientos, Stitch (Taller 01) para prototipar la interfaz, AI Studio (Taller 02) para generar la app funcional con su base de datos y lógica, y Render + GitHub (Taller 03) para desplegarla. Eso cubre las dos fases del PDF y casi todos los entregables del repositorio.

Hay un punto que vale la pena tener presente. El stack que enseñan los talleres (AI Studio genera típicamente apps en React/TypeScript que corren en Node) gravita hacia un frontend que llama directo a la API de Gemini, mientras que el PDF del proyecto pide explícitamente una arquitectura con backend separado y base de datos SQL relacional (DDL y DML). Para Lucía, los datos son inherentemente relacionales: clientes, proyectos, sesiones de tiempo cronometradas, facturas y pagos. Habrá que asegurarse de que lo que construyas no sea solo una interfaz bonita, sino algo con un esquema SQL real detrás, que es donde el caso de Lucía tiene su verdadera sustancia (poder demostrar quién le debe desde cuándo).

Teniendo esto claro, ahora cuál sería el paso a paso para poder realizar este caso y poder entregar lo requerido por el profesor.

Paso 1 — Investigación y requerimientos con NotebookLM

Abres NotebookLM, creas un cuaderno, y con Deep Research generas el informe que alimentará todo lo demás. El prompt debe estar centrado en el dominio de Lucía: gestión del tiempo para freelancers, modelos de facturación mixta (por proyecto, por hora, por suscripción), tarifa efectiva por hora, y herramientas tipo Toggl/Harvest/Bonsai como benchmark. De ahí extraes los pain points, los requerimientos funcionales (registrar clientes, proyectos, cronómetro, facturas, pagos; dashboard con las tres preguntas del caso) y los no funcionales.

El producto de este paso es el **informe ejecutivo en PDF** que va en /docs. Es el documento que justifica todo el sistema, así que merece tu atención temprana.

Paso 2 — Prototipo en Stitch

Con el informe listo, le pides a Stitch que diseñe la interfaz. Para Lucía necesitas como mínimo estas vistas: dashboard financiero (las tres preguntas), lista de clientes, vista de proyecto con el cronómetro, registro/historial de pagos, y generación de factura. Iteras hasta que te guste, creas el prototipo navegable, y guardas el **enlace público de Stitch** para /docs.

Paso 3 — La base de datos (esto es lo que más cuida el profesor)

Aquí está el corazón del caso y donde el PDF es más exigente. Necesitas un script .sql con DDL (estructura) y DML (datos de prueba) en /db. El modelo relacional para Lucía sería aproximadamente: tabla clientes, tabla proyectos (ligada a cliente, con tipo de cobro), tabla sesiones_tiempo (las horas cronometradas, ligadas a proyecto), tabla facturas, y tabla pagos. Ese esquema es lo que permite responder "cuánto me debe este cliente desde enero", que es justamente lo que Lucía no podía demostrar.

Puedes generarlo con Claude o Gemini, pero revísalo tú: que las llaves foráneas tengan sentido y que los datos de prueba reflejen el caso (seis clientes fijos, distintos tipos de cobro, algún pago pendiente desde enero para mostrar el problema resuelto).

Paso 4 — La aplicación con AI Studio (backend + frontend)

Siguiendo el Taller 02, en Google AI Studio generas la app a partir de un prompt que incorpore el informe y, si puedes, los recursos exportados de Stitch. Defines tu stack y lo justificas brevemente (el PDF lo pide). Descargas el .zip y lo llevas a StackBlitz para verlo correr, con npm install y la API Key en .env.local. El código va en /backend y /frontend.

Sé honesto contigo sobre el alcance: una app que muestre el dashboard con las tres preguntas y permita el cronómetro básico es suficiente para demostrar funcionamiento en el video. No persigas todas las funcionalidades.

Paso 5 — Despliegue

El Taller 03 enseña a publicar con GitHub + Render. **El PDF no lo exige como entregable** (pide código en el repo y un video, no una URL en vivo). Si te sobra tiempo, desplégalo porque queda muy bien en el video; si vas justo, sáltalo sin culpa.

Paso 6 — Bitácora de prompts

Mientras haces todo lo anterior, **ve copiando los prompts que funcionaron** (los de NotebookLM, Stitch, AI Studio, las correcciones). El PDF pide evidenciar cómo iteraste con la IA. Es un entregable barato si lo vas guardando sobre la marcha, y carísimo si lo dejas para el final e intentas reconstruirlo de memoria. Va en /docs como Markdown o PDF.

Paso 7 — Repositorio y video

Subes todo a GitHub con la estructura exacta de carpetas (/docs, /db, /backend, /frontend). Grabas el video de máximo 5 minutos mostrando la app funcionando y explicando

brevemente. **Atención al detalle del PDF:** si el video no está disponible al momento de revisar, no hay puntaje, así que súbelo a YouTube o Drive sin restricciones de acceso y verifica el enlace. Solo un integrante hace la entrega por Campus Virtual.

Vamos a empezar con la primera fase del trabajo y posteriormente vamos a pasar a la siguiente. Así que ¿me podrías ayudar con lo primero?

Perfecto. La Fase 1 tiene dos partes: primero la investigación con NotebookLM (que produce el informe), y después el prototipo en Stitch. Empecemos por NotebookLM, que es el insumo de todo lo demás.

El proceso, siguiendo el Taller 01, es: entras a NotebookLM, creas un cuaderno nuevo, pegas un prompt de investigación con Deep Research activado, esperas a que termine y lo importas como fuente. Luego generas un informe a partir de esas fuentes. Te preparo el prompt de investigación adaptado al caso de Lucía.

Este es el prompt para pegar en NotebookLM con **Deep Research** activado:

Contexto: Estoy desarrollando una aplicación de gestión de trabajo y finanzas para diseñadores y profesionales freelance, y necesito un análisis exhaustivo basado en las fuentes que encuentres en la web. El usuario objetivo es una diseñadora gráfica independiente que cobra de tres formas distintas (por proyecto, por hora y por suscripción mensual), no logra llevar control de sus horas ni de los pagos pendientes, y no puede demostrar cuánto le deben sus clientes. Organiza la información en estos tres pilares estratégicos:

1. Gestión del tiempo y productividad freelance: Sintetiza investigaciones sobre time-tracking (registro de tiempo) y su impacto en la rentabilidad del trabajador independiente. Analiza el concepto de "tarifa efectiva por hora" (effective hourly rate): cómo se calcula, por qué difiere de la tarifa nominal, y cómo el trabajo no facturado erosiona los ingresos reales. Extrae evidencia sobre los errores más comunes de los freelancers al cobrar y registrar su trabajo, y sobre cómo la falta de registro sistemático genera pérdidas económicas y conflictos con clientes.

2. Benchmarking de aplicaciones líderes de gestión freelance: Identifica las funcionalidades "core" de las apps líderes del rubro (ej. Toggl Track, Harvest, Bonsai, Clockify, FreshBooks). Analiza cómo manejan el cronómetro de tiempo, la asociación de horas a clientes y proyectos, la facturación a partir del tiempo registrado, y el seguimiento de pagos y deudas pendientes. Determina qué patrones de interfaz (UI) reducen la fricción al registrar tiempo y qué métricas de dashboard consideran indispensables (ingresos, horas por cliente, rentabilidad por proyecto).

3. "Pain Points" y expectativas del usuario freelance: Basado en reviews y foros, identifica los problemas críticos que llevan a los freelancers a abandonar estas herramientas (ej. registro manual tedioso, olvidar iniciar el cronómetro, facturación desconectada del tiempo real). Enumera las funcionalidades que los freelancers consideran indispensables para tener claridad financiera: saber cuánto han facturado, cuánto les deben, qué cliente es más rentable y cuál consume más horas, y cuál es su tarifa efectiva por hora real.

Output deseado: Presenta los hallazgos con una sección de "Insights Accionables" al final de cada pilar, conectando la teoría con funcionalidades prácticas y concretas para la aplicación.

Fíjate que mantuve la misma estructura de tres pilares del prompt que usaron en el Taller 01 con el restaurante, pero traducida al dominio de Lucía. Los tres pilares mapean directamente a las tres preguntas que el dashboard debe responder, lo cual te asegura que la investigación apunte exactamente a lo que el caso exige.

Una vez que NotebookLM termine el Deep Research y lo importes, el segundo prompt (para generar el informe que le entregarás a Stich) sería algo así:

Según las investigaciones sobre gestión del tiempo freelance, los aprendizajes de las apps líderes del rubro y las necesidades del usuario, lista un conjunto de funcionalidades para que mi app —llamada [el nombre que elijas]— resuelva el desorden de una diseñadora freelance que gestiona clientes, proyectos, horas y pagos. Tu tarea es desarrollar un informe para un diseñador web que va a prototipar la app ideal basándose en las instrucciones de este informe. La app debe incluir un dashboard financiero que responda: cuánto he facturado y cuánto me deben, qué cliente me genera más ingresos y cuál me consume más horas, y cuál es mi tarifa efectiva por hora real. También debe permitir iniciar y detener un cronómetro asociado a cada cliente o proyecto.

Quiero que modifiques el documento adjunto y que, básicamente, verifiques las fuentes de los datos que se presentan al principio. Además de eso, quiero que también ancles lo que sale en el informe con la app de la historia concreta de Lucía, para que, básicamente, al momento de llevarlo a Stich, este tenga la capacidad de comprender de mejor manera lo que se le está pidiendo.

Verificación de cifras. Busqué cada estadística del original. El resultado:

La cifra de "\$50,000 anuales por profesional" no tiene respaldo verificable; las fuentes reales hablan de un rango de 10–40% de horas facturables perdidas, equivalente a unos USD 15.600–26.000 al año según la tarifa. La reemplacé por ese rango documentado. Las cifras de "50 millones de horas anuales" y "\$7.4 mil millones diarios" no aparecen en ninguna fuente fiable, así que las retiré. El "69%" sí existe (proviene de un reporte de Ebillity), pero refiere a trabajadores en general, no solo freelancers, y así lo aclaré en la nota al pie. El "92% de errores de facturación" estaba mal: lo correcto, según APQC, es que **más del 60%** de los errores de facturación vienen de la entrada manual de datos, que es más conservador y más defendible. Agregué una sección de fuentes al final y una nota inicial que transparenta que las cifras fueron revisadas, lo cual te protege si el profesor pregunta de dónde salen.

Anclaje al caso de Lucía. Reescribí el documento para que en todo momento hable de la historia concreta y no de un freelancer abstracto. La sección 1 ahora arranca con la situación real de Lucía (seis clientes fijos, facturas en Word, horas en las notas del celular, Excel desactualizado, el cliente que le debía desde enero sin poder demostrarlo). El dashboard (sección 2) está explícitamente organizado alrededor de las tres preguntas del caso. Los tres modelos de cobro se nombran como "los tres modos reales de Lucía". Y la jerarquía de datos

menciona "los 6 fijos + esporádicos". Esto le da a Stitch un contexto mucho más específico para entender qué construir.

Una mejora extra para Stitch. Añadí en la sección 6 una lista priorizada de las cinco vistas a prototipar, marcando el dashboard y el cronómetro como lo imprescindible. Así, cuando lleves el documento a Stitch, la herramienta tiene una guía clara de qué pantallas generar primero, en vez de tener que inferirlas.

Una cosa que decidí por ti, dímelo si prefieres otra cosa: mantuve la distinción entre lo que es "visión de producto" (detección de inactividad, captura de trabajo invisible) y lo que es el prototipo mínimo, dejando lo primero marcado explícitamente como fuera de alcance.

Ahora quiero que diseñes el prompt para utilizarlo en stitch y entregarle, en este caso, el documento que me fue generado por notebook LM para que me genere la aplicación.

Antes de darte el prompt, una aclaración sobre cómo funciona Stitch que conviene tener clara: según el Taller 01, Stitch trabaja principalmente a partir de lo que escribes en el prompt. No siempre acepta adjuntar un PDF como fuente del modo en que lo hace NotebookLM. Por eso te preparo el prompt de dos maneras, y tú eliges según lo que la interfaz de Stitch te permita en el momento:

Opción A — si Stitch te deja adjuntar o pegar el documento: usas un prompt corto que apunta al contenido del informe.

Opción B — si solo acepta texto en el campo de prompt: usas un prompt autocontenido que ya lleva incorporado lo esencial del informe, para que no dependa de un adjunto.

Recuerda del Taller 01 el flujo: escribes el prompt, seleccionas **Aplicación** (no Web), y eliges el modelo **3.1 Pro**.

Opción A — Prompt corto (si adjuntas el informe)

Adjunto el informe de especificaciones funcionales de mi aplicación, llamada **Lúcida**. Es un sistema de gestión para una diseñadora gráfica freelance en Chile que necesita ordenar clientes, proyectos, horas trabajadas y pagos. Diseña la app móvil siguiendo fielmente las funcionalidades y la jerarquía de datos del informe, priorizando las cinco vistas que el documento marca como esenciales: dashboard financiero, cronómetro activo, detalle de cliente, lista de proyectos y generación de factura. Usa un estilo limpio, profesional y cálido, apropiado para una diseñadora. Moneda en pesos chilenos (CLP).

Opción B — Prompt autocontenido (solo texto)

Diseña una aplicación móvil llamada **Lúcida**, un sistema de gestión para una diseñadora gráfica freelance en Chile. Hoy ella cobra de tres formas (por proyecto, por hora y por suscripción mensual), anota sus horas en el celular y lleva los pagos en un Excel desactualizado; por eso una vez no pudo demostrar que un cliente le debía dinero. Lúcida

resuelve ese desorden. Toda la moneda debe mostrarse en pesos chilenos (CLP, formato \$25.000).

Diseña estas vistas:

1. Dashboard financiero (pantalla principal). Es la vista estrella. Debe responder de un vistazo: cuánto ha facturado este mes y cuánto le deben aún (con desglose Pagado / Pendiente / Vencido); qué cliente le genera más ingresos y cuál le consume más horas (un ranking visual); y cuál es su tarifa efectiva por hora real (EHR), mostrada como un número grande y destacado. Incluye tarjetas de métricas y un gráfico simple.

2. Cronómetro activo. Un cronómetro grande de "un solo clic". Antes de iniciar, permite seleccionar Cliente → Servicio → Proyecto, y un selector obligatorio entre "Facturable" y "No facturable". El cronómetro debe verse persistente y fácil de iniciar/detener.

3. Detalle de cliente. Muestra los datos del cliente, sus ingresos acumulados, las horas que ha consumido, y el listado de sus facturas con estado (pagado, pendiente, vencido, con la fecha de origen de cada deuda).

4. Lista de proyectos. Lista o tablero de los proyectos, cada uno con su cliente, su modelo de cobro (por hora / precio fijo / suscripción) y su estado o fase.

5. Generación de factura. Una vista que arma una factura a partir de las horas facturables registradas en el periodo, lista para enviar.

Estilo: limpio, profesional y cálido, pensado para una diseñadora; buena tipografía, espaciado generoso, una paleta de acento sobria. La app debe sentirse como un "asistente de memoria", no como una herramienta de vigilancia.

Modifica únicamente las pantallas que ya existen. No crees ninguna pantalla nueva, no inventes vistas ni contenido, no uses "imaginar nueva pantalla".

Algunas pantallas tienen una barra de navegación inferior y otras no. Quiero que todas las pantallas existentes tengan la misma barra de navegación inferior, idéntica en diseño, posición y orden de íconos.

La barra inferior debe ser consistente en toda la app: los mismos íconos, las mismas etiquetas y el mismo estilo en cada pantalla. Cada ícono debe enlazar a la pantalla correspondiente que ya existe (por ejemplo: Dashboard, Cronómetro, Clientes, Proyectos, Facturas), de modo que se pueda navegar entre las pantallas actuales tocando la barra.

No cambies el contenido ni el diseño del resto de cada pantalla. Solo agrega o corrige la barra de navegación inferior donde falte, manteniéndola igual a la que ya aparece en las pantallas que sí la tienen.

Modifica únicamente las pantallas que ya existen. **No crees ninguna pantalla nueva, no inventes vistas ni contenido, no uses "imaginar nueva pantalla".**

En la barra de navegación inferior, unifica los cinco íconos para que tengan exactamente el mismo nombre y el mismo ícono en las cinco pantallas. Los nombres deben ser, en este orden:

1. **Panel**
2. **Cronómetro**
3. **Clientes**
4. **Proyectos**
5. **Facturas**

Cada uno de estos cinco íconos y nombres debe aparecer **idéntico en las cinco pantallas**: mismo texto, mismo ícono, mismo estilo, mismo orden y misma posición. No debe haber diferencias de nombre ni de ícono entre una pantalla y otra.

Cada ícono debe seguir enlazando a la pantalla que le corresponde (Panel → pantalla del dashboard, Cronómetro → pantalla del cronómetro, Clientes → pantalla de clientes, Proyectos → pantalla de proyectos, Facturas → pantalla de facturas).

No cambies el contenido ni el diseño del resto de cada pantalla. Solo unifica los nombres y los íconos de la barra de navegación inferior.

Ya guardé el link del stitch en notas para después adjuntarlo al trabajo, y también ya realicé la exportación de los datos a Google AI. De hecho, ahora mismo me aparecen como archivos. ¿Qué es lo que debería hacer ahora?

Perfecto, ya estás oficialmente en la Fase 2. Que AI Studio te muestre los diseños como archivos significa que tiene la base visual de tu app lista para trabajar. Ahora viene el paso central del proyecto: convertir esos diseños en una aplicación funcional con base de datos real.

Antes de darte el prompt, conviene que tengas presente lo que el proyecto exige en esta fase, porque es más que "que se vea bonito". El PDF pide tres cosas: definir y justificar un stack tecnológico, generar la base de datos SQL (con scripts DDL para la estructura y DML para datos de prueba), y desarrollar el backend y el frontend conectados. Tu diseño de Stitch cubre el frontend; lo que falta es darle sustancia: la base de datos y la lógica detrás.

El prompt que sigue le pide a AI Studio que construya la app a partir de los diseños que ya tiene, con su base de datos. Lo escribí para que genere algo coherente con el caso de Lucía:

Tengo los diseños de mi aplicación **Lúcida**, un sistema de gestión para una diseñadora gráfica freelance en Chile. Quiero convertir estos diseños en una aplicación web funcional y responsive.

Contexto del negocio: Lucía gestiona seis clientes fijos y varios esporádicos. Cobra de tres formas según el cliente: por hora, por precio fijo (proyecto) y por suscripción mensual. Necesita registrar sus horas con un cronómetro, llevar control de sus clientes y proyectos, y saber cuánto ha facturado, cuánto le deben y cuál es su tarifa efectiva por hora.

La aplicación debe tener una base de datos SQL relacional con estas entidades y relaciones:

- **Clientes** (nombre, contacto, tipo, moneda en CLP).
- **Servicios** (catálogo: Diseño UI, Identidad de marca, Mantenimiento, etc.).
- **Proyectos** (vinculados a un cliente y un servicio; con modelo de cobro: por hora, precio fijo o suscripción; y un estado).
- **Sesiones de tiempo** (registros del cronómetro, vinculados a un proyecto, con marca de facturable o no facturable, fecha y duración).
- **Facturas** (vinculadas a un cliente/proyecto, con monto, fecha de emisión, fecha de origen de la deuda y estado: pagada, pendiente o vencida).

Funcionalidades clave:

- Un **dashboard** que muestre: total facturado del mes y total por cobrar (desglosado en pagado, pendiente y vencido); ranking de clientes por ingresos y por horas consumidas; y la tarifa efectiva por hora ($EHR = \text{ingresos totales} \div (\text{horas facturables} + \text{no facturables})$).
- Un **cronómetro** que se asocie a cliente, servicio y proyecto, con selección facturable/no facturable.
- Vistas de **clientes, proyectos y facturas**.

Toda la moneda debe mostrarse en pesos chilenos (formato \$25.000).

Construye la app respetando los diseños que ya cargué. Usa un stack apropiado y dime cuál elegiste y por qué.

En el dashboard, la tarjeta del cronómetro muestra el texto "play_arrow" en vez del ícono de reproducir. El nombre del ícono se está mostrando como texto. Corrige esto para que se vea el ícono de play (▶) correctamente, no la palabra. Revisa también el resto de la app por si hay otros íconos que muestren su nombre en texto en lugar del ícono. No cambies nada más

Ya inserté el prompt del arreglo del cronómetro, y de hecho, ya funciona bien con el logo. Sin embargo, me di cuenta de otra problemática, y es que cuando creo algo no me deja eliminarlo. Aparece el ícono de eliminar (ícono rojo con un símbolo de basura), pero no elimina nada. ¿Me podrías ayudar con ese tema?

En la aplicación, el botón de eliminar (el ícono rojo de basurero) no funciona: al presionarlo no se borra el elemento. Corrige la funcionalidad de eliminar para que funcione correctamente en todas las secciones donde aparece (clientes, proyectos, facturas y registros del cronómetro).

Al presionar eliminar, el elemento debe borrarse de verdad, tanto de la vista como de la base de datos SQL. Agrega un mensaje de confirmación antes de borrar ("¿Estás seguro de que quieres eliminar esto?") para evitar borrados accidentales. Después de eliminar, la lista y el dashboard deben actualizarse para reflejar el cambio.

Ten en cuenta las relaciones de la base de datos: por ejemplo, si se elimina un cliente que tiene proyectos o facturas asociadas, decide un comportamiento coherente (impedir el borrado avisando que primero hay que eliminar lo asociado, o eliminar en cascada) y aplícalo de forma consistente. No cambies nada más de la aplicación.

Ahora se arregló ese problema, sin embargo, me di cuenta de otro, y es que cuando ocupo el cronómetro, me da la opción de pausar, pero no me deja pausar, no me deja presionar el botón, te adjunto una foto del botón, es el botón verde, no me deja ocuparlo. O sea, cuando lo ocupo, lo presiono el botón verde, sí me deja ocuparlo, pero cuando lo aprieto pasa a rojo, al pasarlo a rojo no puedo detenerlo.

En el cronómetro de "Registrar Sesión de Tiempo" hay un problema: el botón verde "Iniciar Cronómetro" funciona y el cronómetro empieza a correr (el botón cambia a rojo), pero al presionar el botón rojo para pausar o detener, no pasa nada: no responde.

Antes de corregir, diagnostica paso a paso: revisa si el botón en estado rojo (detener/pausar) tiene un evento de clic conectado, si ese evento llama a la función que detiene el cronómetro, y si esa función efectivamente para el conteo del tiempo. Identifica en cuál paso se rompe.

Luego corrige el punto exacto donde falla, para que el botón rojo detenga o pause el cronómetro correctamente, dejando el tiempo registrado para poder guardarlo. Explicame qué estaba causando el problema y qué cambiaste.

Ya realicé la conexión con GitHub y salió todo correcto. Sin embargo, me surgió una duda antes de empezar con el Excel, y es que cuando yo salgo o cierro mi cuenta, o inicio sesión otra vez, ya no me deja ocupar la opción de generar esos datos "prueba". ¿Podrías crear un prompt para que cuando yo inicie la sesión siempre pueda volver a ocupar esos datos prueba?

En la aplicación, el botón "Cargar datos demo" desaparece después de usarlo una vez y ya no vuelve a aparecer cuando reabro la sesión. Quiero que ese botón **esté siempre disponible**, sin importar si la app ya tiene datos o no.

Modifica la lógica para que el botón "Cargar datos demo" se muestre siempre (por ejemplo, en el dashboard o en un menú de opciones), de modo que yo pueda volver a cargar los datos de demostración cuando quiera.

Para evitar duplicados al recargar: antes de insertar los datos demo, que la función limpie o reemplace los datos demo anteriores, de manera que al presionar el botón la app vuelva siempre a un estado de demostración limpio y consistente. No cambies nada más de la aplicación.

Antes de seguir, tengo una pequeña duda, y es que cuando se plantea el caso, se dice que Lucía básicamente, pues, maneja todo desde Word, Excel, notas, etcétera. Sin embargo, por ejemplo, si el profesor me preguntara de poder adjuntar un archivo Excel con los datos, la aplicación no me deja adjuntarlo. ¿Es muy avaricioso lo que estoy pidiendo?, o podría, por ejemplo, la aplicación también admitir datos externos nuevos, ya sea Excel, Word, etcétera. En este caso, es más probable que sea Excel, y básicamente poder añadirlos a la aplicación.

Quiero agregar a la aplicación la posibilidad de importar datos desde archivos Excel (.xlsx) o CSV, para que Lucía pueda cargar la información que hoy tiene en sus planillas sin reescribirla a mano. Vamos a construir esto por etapas, empezando por los clientes.

Agrega en la sección de Clientes un botón "Importar desde Excel" que permita:

1. Subir un archivo .xlsx o .csv.
2. Leer las columnas (nombre, contacto, tipo de cliente) y mostrarme una vista previa de lo que se va a importar antes de confirmar.
3. Al confirmar, insertar esos clientes en la base de datos SQL.
4. Si una fila tiene un error o un dato faltante, indicarme exactamente qué fila y qué problema, en lugar de fallar en silencio.

Diseña la función de importación de forma **modular y reutilizable**, porque después voy a extender esta misma funcionalidad a Proyectos y Facturas. Mantén todo lo demás de la aplicación igual. Al terminar, indícame qué columnas exactas debe tener el archivo Excel para que la importación funcione.

Ahora vamos a extender la importación desde Excel a la sección de **Proyectos**, usando la misma función modular que ya creamos para Clientes.

Agrega en la sección de Proyectos un botón "Importar desde Excel" que permita subir un .xlsx o .csv y muestre la misma vista previa con validación fila por fila antes de confirmar.

Los proyectos deben incluir estos campos:

- **Nombre del proyecto** (obligatorio).
- **Cliente** (obligatorio): debe corresponder a un cliente que ya existe en la base de datos. Si el cliente del Excel no existe, marca esa fila como error indicando "el cliente X no existe", en lugar de crearlo.
- **Servicio** (por ejemplo: Diseño UI, Identidad de marca, Mantenimiento).
- **Modelo de cobro** (obligatorio): debe ser uno de estos tres valores: "Por hora", "Precio fijo" o "Suscripción". Normaliza variantes de escritura igual que hiciste con el tipo de cliente.
- **Estado** (opcional, por ejemplo: Briefing, Diseño, Feedback, Entrega; si viene vacío, asigna un estado por defecto).

Mantén el mismo comportamiento que en Clientes: importar solo las filas válidas, reportar las filas con error indicando fila y problema, y normalizar mayúsculas/espacios. Al terminar, indícame qué columnas exactas debe tener el Excel de proyectos.

No cambies nada más de la aplicación.

Columnas exactas que debe tener el archivo Excel/CSV para Proyectos

El importador está diseñado con un algoritmo de mapeo flexible que reconoce sinónimos (en mayúsculas, minúsculas o con tildes). No obstante, para máxima precisión, se recomienda emplear los nombres de columna del siguiente estándar:

Nombre del Proyecto (Obligatorio)

Nombre recomendado de columna: Nombre del Proyecto, Nombre, Proyecto o Project

Descripción: Texto identificativo del proyecto.

Cliente (Obligatorio)

Nombre recomendado de columna: Cliente, Client, Customer o Empresa

Descripción: El nombre exacto de la empresa o cliente. Debe coincidir con un cliente que ya exista en la base de datos (la comparación es insensible a mayúsculas/minúsculas y espacios adicionales). Si el cliente no existe en la base de datos, el importador marcará la fila con un error detallativo ("El cliente X no existe en la base de datos") y omitirá el registro asegurando la integridad estructural.

Servicio (Opcional)

Nombre recomendado de columna: Servicio, Service, Catálogo o Tipo de servicio

Descripción: El tipo de servicio brindado (por ejemplo, Diseño UI, Identidad de marca, Mantenimiento). Si la categoría no existe en el catálogo registrado, Lúcida la crea automáticamente en tiempo de ejecución para añadirla al catálogo de servicios. Si se deja vacía, se asocia por defecto a la primera especialidad disponible (Diseño UI).

Modelo de Cobro (Obligatorio)

Nombre recomendado de columna: Modelo de Cobro, Modelo, Cobro o Billing Model

Descripción: Debe corresponder a uno de los tres modelos de honorarios soportados:

'Por hora' (también reconoce 'hora', 'hour')

'Precio fijo' (también reconoce 'fijo', 'fixed', 'precio')

'Suscripción' (también reconoce 'suscrip', 'subs', 'abono', 'mensual')

Normaliza automáticamente variantes de escritura a estos valores.

Tarifa (Opcional)

Nombre recomendado de columna: Tarifa, Rate, Costo, Precio o Fee

Descripción: Número entero mayor o igual a 0. Representa el precio nominal (por hora, coste total, o abono mensual dependiendo del modelo). Si viene vacío o contiene caracteres de moneda (como \$), se limpia automáticamente y asume 0.

Estado (Opcional)

Nombre recomendado de columna: Estado, Status, Etapa o Stage

Descripción: Fase en la que se encuentra el proyecto (por ejemplo: Briefing, Diseño, Feedback, Entrega, Activo, Completado, o Pausado). Si se omite, se asigna automáticamente el estado por defecto 'Activo'.

Detalles técnicos & Arquitectura de la solución

Flexibilidad & Pruebas Unitarias Robustas: El componente ExcelImporter.tsx procesa el Excel, asocia las columnas mediante sinónimos y realiza validaciones en memoria.

Diagnóstico Exhaustivo: Si el archivo tiene filas con nombres de clientes incorrectos, campos obligatorios ausentes o montos de tarifas inválidos, la aplicación presentará una alerta visual de diagnóstico indicando la fila exacta y el problema específico, permitiéndote corregirlo mientras se importan con seguridad todos los demás registros válidos.

Persistencia Dinámica de Relaciones: El flujo mapea las claves foráneas con el ID del cliente correspondiente e interactúa dinámicamente con el catálogo de servicios, asegurando que se cumplan todas las restricciones de integridad de la base de datos PostgreSQL.

Ahora vamos a extender la importación desde Excel a la sección de **Facturas**, usando la misma función modular de Clientes y Proyectos.

Agrega en la sección de Facturas un botón "Importar desde Excel" con la misma vista previa y validación fila por fila antes de confirmar.

Las facturas deben incluir estos campos:

- **Cliente** (obligatorio): debe corresponder a un cliente que ya existe en la base de datos. Si no existe, marca la fila como error.
- **Proyecto** (opcional): si se indica, debe corresponder a un proyecto existente de ese cliente; si no existe, marca error.

- **Monto** (obligatorio): número entero ≥ 0 ; limpia símbolos de moneda (\$) y puntos de miles igual que en la tarifa de proyectos.
- **Fecha de emisión** (obligatorio): fecha en que se emitió la factura.
- **Fecha de origen de la deuda** (opcional): la fecha desde la cual se debe el dinero (esto es clave para el caso: poder mostrar "pendiente desde enero").
- **Estado** (obligatorio): debe ser uno de estos tres valores: "Pagada", "Pendiente" o "Vencida". Normaliza variantes de escritura.

Mantén el mismo comportamiento: importar solo filas válidas, reportar errores con fila y problema, normalizar mayúsculas/espacios/moneda. El monto importado debe reflejarse correctamente en el dashboard (total facturado, por cobrar, y los desgloses pagado/pendiente/vencido). Al terminar, indícame qué columnas exactas debe tener el Excel de facturas.

No cambies nada más de la aplicación.

He completado exitosamente la implementación de la característica de importación desde Excel/CSV para la sección de **Facturas**, integrando y reutilizando el componente modular de previsualización y validación por filas.

Resumen del Trabajo Realizado

1. Estructura de Datos e Inserción:

- Agregué el botón **Importar desde Excel** dentro de la pestaña de **Facturas**, conservando exactamente el diseño visual y la paleta de colores de la aplicación.
- Definí la estructura de mapeo de columnas `invoiceColumnConfig` con robustas reglas de sinonimia lingüística para el emparejamiento automático de cabeceras en español e inglés.

2. Validaciones por fila y en cascada:

- **Cliente (Obligatorio)**: Busca coincidencias del nombre de forma "insensible a mayúsculas/minúsculas" contra la base de datos de clientes existentes. Si no se localiza, se marca la fila como error indicando exactamente que el cliente no existe.
- **Proyecto (Opcional)**: Enriquecí el proceso de validación en el componente `ExcelImporter` para permitir la validación contextual por filas. Si se proporciona un proyecto, verifica que pertenezca exclusivamente al cliente de esa misma fila y que exista en el sistema.
- **Monto (Obligatorio)**: Limpia de forma segura caracteres no numéricos como signos de divisa (\$), puntos, comas o espacios, y valida que sea un número entero mayor o igual a 0.
- **Fechas**: Implementé un formateador superseguro (`parseImportDate`) capaz de interpretar fechas en formato de texto estándar (YYYY-MM-DD, DD/MM/YYYY, DD-MM-YYYY) o códigos seriales de Excel. La **Fecha de origen de la deuda** es opcional; si viene vacía, se hereda automáticamente el valor de la **Fecha de emisión**.

- **Estado (Obligatorio):** Normaliza variantes terminológicas como "Pagada" (pagada), "Pendiente" (pendiente) o "Vencida" (vencida).
- 3. Garantía y Compatibilidad Financiera:**
- Puesto que los indicadores del dashboard y resúmenes del menú lateral se calculan dinámicamente en base a todas las facturas de la base de datos, las facturas importadas exitosamente alimentan automáticamente el cálculo del **Total Facturado**, deudas **Pendientes**, **Vencidas** y flujos generales en tiempo real.

Columnas Exactas requeridas para el Excel de Facturas

Para que Lúcida reconozca y procese tu planilla de facturas de forma automática, asegúrate de que tu hoja de cálculo contenga columnas con cualquiera de las siguientes cabeceras (no importa el orden ni el uso de mayúsculas o tildes):

Campo	Cabeceras Recomendadas (Sinónimos Aceptados)	Requerido	Formato / Valores aceptados
Cliente	Cliente, Client, Customer, Empresa, Razon Social	Sí (Obligatorio)	Texto (El cliente ya debe estar creado en Lúcida).
Proyecto	Proyecto, Project, Nombre del proyecto	No (Opcional)	Texto (Debe coincidir con un proyecto de ese cliente).
Monto	Monto, Amount, Total, Valor, Pesos, Cobro, Precio	Sí (Obligatorio)	Número o texto formateado (ej. 1.850.000 o \$250000).
Fecha Emisión	Fecha Emision, Fecha de Emision, Date, Issue Date	Sí (Obligatorio)	Fecha legible (YYYY-MM-DD, DD/MM/YYYY o fecha nativa).
Origen de Deuda	Origen de deuda, Fecha origen, Deuda desde, Desde	No (Opcional)	Fecha legible (Si se omite, se iguala a la Fecha Emisión).
Estado	Estado, Status, Estado de pago, Pago	Sí (Obligatorio)	Pagada (o <i>Paid</i>), Pendiente (o <i>Due</i>), Vencida (o <i>Expired</i>).

Cuando tú hablas del script SQL, estás hablando de también el tema del ddl y dml, ¿o no? ¿Qué es lo que pide al inicio de la fase dos? Si es así, entrégame el prompt para yo poder pedírselo a Google AI Studio y poder básicamente insertarlo en GitHub.

Necesito exportar la base de datos completa de la aplicación en un único script .sql de PostgreSQL, para incluirlo como entregable del proyecto. El archivo debe contener las dos partes:

1. **DDL (estructura):** las sentencias CREATE TABLE de todas las tablas (clientes, servicios, proyectos, sesiones de tiempo, facturas y cualquier otra que exista), con sus tipos de datos, llaves primarias, llaves foráneas y las relaciones entre ellas.
2. **DML (datos de prueba):** las sentencias INSERT con los datos que ya están cargados en la app (los clientes, proyectos y facturas, incluyendo la factura vencida desde enero), para el poblamiento inicial.

Entrégamelo todo en un solo bloque de código SQL, en el orden correcto de ejecución: primero los CREATE TABLE (las tablas sin dependencias antes que las que tienen llaves

foráneas), y luego los INSERT en orden (primero clientes y servicios, después proyectos, después facturas y sesiones de tiempo), de modo que el script se pueda ejecutar de corrido sin errores de integridad referencial. Debe ser SQL válido para PostgreSQL, listo para guardar como un archivo llamado lucida.sql.

```
--  
=====
```

```
-- BASE DE DATOS LÚCIDA - SCRIPT DE CREACIÓN Y POBLAMIENTO (lucida.sql)
```

```
-- Sistema de Gestión para Diseñadoras Freelance
```

```
-- Caso: "Freelance en Apuros - La vida desordenada de Lucía"
```

```
-- Motor: PostgreSQL
```

```
-- Contiene: DDL (estructura de tablas) y DML (datos de prueba)
```

```
--  
=====
```

```
-- =====
```

```
-- 0. ELIMINACIÓN DE TABLAS EXISTENTES (permite re-ejecución limpia)
```

```
-- =====
```

```
DROP TABLE IF EXISTS facturas CASCADE;
```

```
DROP TABLE IF EXISTS sesiones_tiempo CASCADE;
```

```
DROP TABLE IF EXISTS proyectos CASCADE;
```

```
DROP TABLE IF EXISTS servicios CASCADE;
```

```
DROP TABLE IF EXISTS clientes CASCADE;
```

```
DROP TABLE IF EXISTS users CASCADE;
```

```
-- =====
```

```
-- 1. ESTRUCTURA (DDL) EN ORDEN DE RELACIÓN
```

```
-- =====
```

```
-- Tabla de Usuarios (la dueña del sistema: Lucía)
```

```
CREATE TABLE users (
```

```
    id SERIAL PRIMARY KEY,
```

```
    uid VARCHAR(255) NOT NULL UNIQUE,
```

```
    email VARCHAR(255) NOT NULL,
```

```
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```
);
```

```
-- Tabla de Clientes
```

```
CREATE TABLE clientes (
```

```
    id SERIAL PRIMARY KEY,
```

```
user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  
nombre VARCHAR(255) NOT NULL,  
  
contacto VARCHAR(255) NOT NULL,  
  
tipo VARCHAR(50) NOT NULL, -- 'Fijo' o 'Esporádico'  
  
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

-- Tabla de Catálogo de Servicios

```
CREATE TABLE servicios (  
  
id SERIAL PRIMARY KEY,  
  
user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  
nombre VARCHAR(255) NOT NULL,  
  
descripcion TEXT,  
  
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

-- Tabla de Proyectos

```
CREATE TABLE proyectos (  
  
id SERIAL PRIMARY KEY,  
  
user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  
cliente_id INTEGER NOT NULL REFERENCES clientes(id) ON DELETE CASCADE,  
  
servicio_id INTEGER NOT NULL REFERENCES servicios(id) ON DELETE  
RESTRICT,  
  
nombre VARCHAR(255) NOT NULL,
```

```
modelo_cobro VARCHAR(100) NOT NULL, -- 'Por hora', 'Precio fijo', 'Suscripción'

tarifa INTEGER NOT NULL DEFAULT 0, -- Tarifa del proyecto en CLP

estado VARCHAR(100) NOT NULL,      -- Estado de fase/flujo de trabajo

created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP

);
```

-- Tabla de Sesiones de Registro de Tiempo (cronómetro)

```
CREATE TABLE sesiones_tiempo (

    id SERIAL PRIMARY KEY,

    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    proyecto_id INTEGER NOT NULL REFERENCES proyectos(id) ON DELETE CASCADE,

    facturable BOOLEAN NOT NULL DEFAULT TRUE,

    fecha VARCHAR(10) NOT NULL,      -- Formato YYYY-MM-DD

    duracion INTEGER NOT NULL,       -- Duración en segundos

    descripcion TEXT,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP

);
```

-- Tabla de Facturas Emitidas

```
CREATE TABLE facturas (

    id SERIAL PRIMARY KEY,

    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    cliente_id INTEGER NOT NULL REFERENCES clientes(id) ON DELETE CASCADE,
```

```
    proyecto_id INTEGER REFERENCES proyectos(id) ON DELETE SET NULL, --
Relación opcional
```

```
    monto INTEGER NOT NULL, -- Monto en CLP
```

```
    fecha_emision VARCHAR(10) NOT NULL, -- Formato YYYY-MM-DD
```

```
    fecha_origen_deuda VARCHAR(10) NOT NULL, -- Formato YYYY-MM-DD
(antigüedad de deuda)
```

```
    estado VARCHAR(50) NOT NULL, -- 'pagada', 'pendiente', 'vencida'
```

```
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```
);
```

```
-- =====
```

```
-- 2. DATOS DE PRUEBA (DML) EN ORDEN DE INTEGRIDAD
```

```
-- =====
```

```
-- 2.1 Usuario (Lucía, dueña del sistema)
```

```
INSERT INTO users (id, uid, email) VALUES
```

```
(1, 'lucida-user-01', 'lucia.freelance@lucida.cl');
```

```
-- 2.2 Catálogo de Servicios
```

```
INSERT INTO servicios (id, user_id, nombre, descripcion) VALUES
```

```
(1, 1, 'Diseño UI', 'Diseño de interfaces web y móviles'),
```

```
(2, 1, 'Identidad de marca', 'Logos, paletas de colores y manuales de marca'),
```

```
(3, 1, 'Mantenimiento', 'Soporte y actualización continua de piezas de diseño'),
```

(4, 1, 'Ilustración', 'Ilustraciones personalizadas y assets vectoriales'),

(5, 1, 'Diseño Web', 'Maquetación y diseño de sitios corporativos y landing pages'),

(6, 1, 'Consultoría', 'Asesoría estratégica de marca y diseño');

-- 2.3 Clientes (6 fijos + esporádicos, según el caso de Lucía)

INSERT INTO clientes (id, user_id, nombre, contacto, tipo) VALUES

(1, 1, 'Editorial Norte', 'contacto@editorialnorte.cl', 'Fijo'),

(2, 1, 'Estudio Andes', 'hola@estudioandes.cl', 'Fijo'),

(3, 1, 'Café Raíz', '+56 9 8765 4321', 'Fijo'),

(4, 1, 'Boutique Lúmina', 'ventas@lumina.cl', 'Fijo'),

(5, 1, 'Constructora Sur', 'proyectos@constructorasur.cl', 'Fijo'),

(6, 1, 'Marca Personal · Javiera', 'javiera.dg@gmail.com', 'Esporádico'),

(7, 1, 'Taller Bicicletas Pedal', 'info@tallerpedal.cl', 'Esporádico');

-- 2.4 Proyectos (vinculados a cliente y servicio, con su modelo de cobro)

INSERT INTO proyectos (id, user_id, cliente_id, servicio_id, nombre, modelo_cobro, tarifa, estado) VALUES

(1, 1, 1, 2, 'Rediseño identidad de marca', 'Precio fijo', 1200000, 'Diseño'),

(2, 1, 1, 3, 'Mantenión web mensual', 'Suscripción', 150000, 'Activo'),

(3, 1, 3, 1, 'App carta digital', 'Por hora', 25000, 'Briefing'),

(4, 1, 4, 1, 'Catálogo de productos', 'Precio fijo', 800000, 'Feedback'),

(5, 1, 5, 2, 'Brandbook corporativo', 'Precio fijo', 1500000, 'Activo'),

(6, 1, 6, 4, 'Ilustraciones redes sociales', 'Suscripción', 90000, 'Activo'),

(7, 1, 2, 6, 'Asesoría de marca', 'Por hora', 30000, 'Activo');

-- 2.5 Sesiones de Tiempo (registros del cronómetro; duración en segundos)

```
INSERT INTO sesiones_tiempo (id, user_id, proyecto_id, facturable, fecha, duracion,
descripcion) VALUES
```

```
(1, 1, 1, TRUE, '2026-06-15', 122400, 'Bocetos y propuestas de identidad'), -- 34 h
```

```
(2, 1, 3, TRUE, '2026-06-16', 86400, 'Maquetado de pantallas de la carta'), -- 24 h
```

```
(3, 1, 7, TRUE, '2026-06-17', 43200, 'Sesión de asesoría y revisión de marca'), -- 12 h
```

```
(4, 1, 2, FALSE, '2026-06-17', 18000, 'Reuniones de coordinación (no facturable)'),-- 5 h
```

```
(5, 1, 5, TRUE, '2026-06-18', 64800, 'Desarrollo del brandbook corporativo'); -- 18 h
```

-- 2.6 Facturas (incluye la deuda vencida desde enero: caso central de Lucía)

```
INSERT INTO facturas (id, user_id, cliente_id, proyecto_id, monto, fecha_emision,
fecha_origen_deuda, estado) VALUES
```

```
(1, 1, 1, 1, 1200000, '2026-03-10', '2026-03-10', 'pagada'),
```

```
(2, 1, 1, 2, 150000, '2026-02-01', '2026-02-01', 'pendiente'),
```

```
(3, 1, 3, 3, 450000, '2026-01-15', '2026-01-15', 'vencida'), -- << pendiente desde enero
```

```
(4, 1, 4, 4, 800000, '2026-03-15', '2026-03-15', 'pendiente'),
```

```
(5, 1, 5, 5, 1500000, '2026-02-20', '2026-02-20', 'pagada'),
```

```
(6, 1, 6, NULL, 90000, '2026-04-01', '2026-04-01', 'pendiente'),
```

```
(7, 1, 2, 7, 300000, '2026-04-05', '2026-04-05', 'pendiente');
```

```
--
=====
=====
```

-- 3. ACTUALIZACIÓN DE SECUENCIAS (corrección de llaves seriales en PostgreSQL)

```
--  
=====
```

```
SELECT setval('users_id_seq', (SELECT MAX(id) FROM users));
```

```
SELECT setval('clientes_id_seq', (SELECT MAX(id) FROM clientes));
```

```
SELECT setval('servicios_id_seq', (SELECT MAX(id) FROM servicios));
```

```
SELECT setval('proyectos_id_seq', (SELECT MAX(id) FROM proyectos));
```

```
SELECT setval('sesiones_tiempo_id_seq', (SELECT MAX(id) FROM sesiones_tiempo));
```

```
SELECT setval('facturas_id_seq', (SELECT MAX(id) FROM facturas));
```

Quiero un botón de reinicio para poder realizar la presentación en video.

Quiero agregar un botón para reiniciar la aplicación a cero, que me sirva para demostraciones: que borre todos los datos (clientes, proyectos, servicios, sesiones de tiempo y facturas) y deje la app limpia, como recién instalada.

Agrega el botón en un lugar discreto (por ejemplo, en un menú de configuración o ajustes), con el texto "Reiniciar datos" o "Empezar de cero". Al presionarlo, debe pedir una confirmación clara antes de borrar ("Esto eliminará todos los datos de forma permanente. ¿Continuar?").

Al confirmar, debe eliminar todos los registros de todas las tablas respetando el orden de las relaciones (primero las facturas y sesiones de tiempo, luego proyectos, luego clientes y servicios) para no romper la integridad de la base de datos. Después, la app debe quedar vacía y el dashboard en cero. No cambies nada más.

Ahora vamos a extender la importación desde Excel a la sección de **Sesiones de Tiempo** (la bitácora del cronómetro), usando la misma función modular de Clientes, Proyectos y Facturas.

Agrega en la sección de Bitácora/Sesiones de tiempo un botón "Importar desde Excel" con la misma vista previa y validación fila por fila antes de confirmar.

Las sesiones deben incluir estos campos:

- **Proyecto** (obligatorio): debe corresponder a un proyecto que ya existe en la base de datos. Si no existe, marca la fila como error.

- **Duración** (obligatorio): la duración de la sesión. Voy a ingresarla en **horas** (por ejemplo 2, 1.5, 40); conviértela internamente a segundos si la base las guarda así.
- **Facturable** (obligatorio): un valor que indique si es facturable o no (acepta "Sí/No", "Facturable/No facturable", "true/false"). Normalízalo.
- **Fecha** (obligatorio): la fecha de la sesión, en formato YYYY-MM-DD o formatos comunes.
- **Descripción** (opcional): texto libre de lo que se hizo.

Mantén el mismo comportamiento: importar solo filas válidas, reportar errores con fila y problema. Las sesiones importadas deben reflejarse en la bitácora y alimentar el cálculo del EHR y el ranking de horas por cliente en el dashboard. Al terminar, indícame qué columnas exactas debe tener el Excel.

No cambies nada más de la aplicación.

He extendido exitosamente la funcionalidad de importación desde planillas Excel a la sección de **Bitácora de Tiempos** (sesiones de tiempo) de la aplicación. Manteniendo la consistencia estética y modular que ya poseían las secciones de Clientes, Proyectos y Facturas, el sistema ahora permite importar registros de manera masiva con validación interactiva fila por fila y normalización inteligente.

Resumen de mejoras implementadas

- **Botón "Importar desde Excel"**: Se ha integrado de manera elegante en el encabezado responsivo de la sección de Bitácora.
- **Validación Fila por Fila**: Se valida que cada fila cumpla con los requisitos necesarios (validación de proyecto existente, fecha válida, duración mayor a cero, etc.) destacando visualmente los errores encontrados sin interrumpir la importación del resto de las filas que sí son correctas.
- **Normalización de Datos**: Se procesan las horas a segundos automáticamente, se estandarizan las variadas opciones del campo "Facturable" a booleans, y se procesan múltiples formatos de fecha.
- **Actualización en Tiempo Real**: Tras confirmar la importación, los nuevos registros se reflejan inmediatamente en el historial y recalculan en tiempo real el **EHR** (Tarifa Horaria Efectiva) y el ranking de horas por cliente del Dashboard.

Columnas exactas y especificaciones de la planilla Excel

Para realizar la importación con éxito, tu hoja de cálculo (.xlsx, .xls o .csv) debe contener las siguientes columnas (el importador es flexible con mayúsculas, minúsculas, espacios y acentos):

1. **Proyecto** (*Obligatorio*):

- **Nombres de columna aceptados:** Proyecto, Nombre de Proyecto, Project, etc.
- **Descripción:** Debe coincidir exactamente con el nombre de un proyecto que ya esté registrado en Lúcida. Si no se encuentra el proyecto, la fila se marcará con error.

2. Duración (Horas) (Obligatorio):

- **Nombres de columna aceptados:** Duración (Horas), Duración, Horas, Hours, Duration, etc.
- **Descripción:** La duración de la sesión expresada en horas decimales (por ejemplo, 2, 1.5, 0.75). Admite números usando punto . o coma , como separador decimal. Internamente, se convertirá de forma automática a segundos.

3. Facturable (Obligatorio):

- **Nombres de columna aceptados:** Facturable, Cobrar, Billable, etc.
- **Descripción:** Determina si los tiempos corresponden a trabajo facturado. Acepta de forma flexible: Sí/No, Facturable/No facturable, true/false, Yes/No o 1/0.

4. Fecha (Obligatorio):

- **Nombres de columna aceptados:** Fecha, Día, Date, etc.
- **Descripción:** La fecha de realización de la actividad. Soporta el formato estándar YYYY-MM-DD, el formato clásico DD/MM/YYYY, fechas relativas o el código serial interno de fechas que genera Excel automáticamente.

5. Descripción (Opcional):

- **Nombres de columna aceptados:** Descripción, Detalle, Notas, Description, etc.
- **Descripción:** Texto libre descriptivo sobre la tarea exacta que se llevó a cabo. Si se deja en blanco, se registrará vacía por defecto.

En el dashboard, el panel "Horas Consumidas por Cliente" no está ordenando correctamente: muestra a Constructora Sur primero (18 horas), pero Café Raíz tiene muchas más horas consumidas (más de 50) y no aparece arriba, ni siquiera se ve en la lista.

Corrige este panel para que: 1) ordene los clientes de mayor a menor según el total de horas consumidas (sumando todas las sesiones de tiempo de los proyectos de cada cliente, facturables y no facturables); 2) Café Raíz, que es el que más horas tiene, aparezca primero. Si el panel solo muestra un número limitado de clientes (top 3 o top 5), asegúrate de que el orden sea por horas totales descendente, de modo que los que más horas consumen sean los que se muestren.

Verifica también que la etiqueta de rentabilidad de cada cliente sea coherente: un cliente con muchas horas no facturables (como Café Raíz) debería reflejar una rentabilidad o tarifa efectiva menor, no aparecer simplemente como "Rentable". No cambies nada más del dashboard.

Diagnostica primero dónde está el problema en el ordenamiento antes de corregir, y explícame qué cambiaste.

Dame un prompt para que sean los dos igual de la misma cantidad, y básicamente porque el recuadro es mucho más largo, siendo que debiera ser incluso más cortito también, por un tema de estética.

En el dashboard, los dos paneles "Ranking Clientes por Ingresos" y "Horas Consumidas por Cliente" muestran distinta cantidad de clientes (uno muestra 4 y el otro 5), y además el recuadro de la izquierda queda con mucho espacio vacío que lo hace verse desproporcionado.

Corrige dos cosas:

1. Que ambos paneles muestren **la misma cantidad de clientes** (top 5 en los dos), cada uno con su criterio de orden: el de ingresos ordenado por monto facturado descendente, y el de horas por horas consumidas descendente.
2. Ajusta la **altura de ambos recuadros** para que se adapten a su contenido y queden visualmente equilibrados entre sí, sin espacio vacío sobrante que los alargue innecesariamente. Los dos paneles deben verse parejos en tamaño, uno al lado del otro.

No cambies los datos ni el resto del dashboard, solo la cantidad de clientes mostrados y el alto/proporción de los recuadros.

La verdad es que me gustaría poder tener la aplicación pública, porque creo que se le da mucho más peso al momento de presentar la aplicación, además de que podría compartirla con mis compañeros de trabajo.

Buena decisión. Se intentará el despliegue siguiendo el Taller 3 (GitHub + Render), advirtiéndole que el caso de la app es más complejo que el ejemplo del taller por usar base de datos PostgreSQL y autenticación. Se acuerda grabar el video desde AI Studio como respaldo antes de arriesgar el despliegue.

Render construyó la app pero falló al arrancar con el error: `TypeError [ERR_INVALID_ARG_TYPE]: The "path" argument must be of type string or an instance of URL. Received undefined`, relacionado con `fileURLToPath(import.meta.url)`. Prompt enviado a AI Studio para corregirlo:

Estoy intentando desplegar mi aplicación en Render y falla al arrancar con este error: `TypeError [ERR_INVALID_ARG_TYPE]: The "path" argument must be of type string or an instance of URL. Received undefined`, relacionado con `fileURLToPath(import.meta.url)`. El build genera el servidor en formato CommonJS (cjs), donde `import.meta.url` no está disponible y queda undefined. Corrige el código del servidor (`server.ts`) para que obtenga la ruta del archivo de una forma compatible con el formato de salida del build, de modo que la app pueda arrancar tanto en local como al desplegarse en Render. Explicame qué cambiaste.

Resolvió el problema adaptando la resolución de rutas con compatibilidad híbrida ESM/CommonJS, reemplazando las variables globales `__filename` y `__dirname` por versiones locales de respaldo para evitar conflictos de alcance, manteniendo compatibilidad con el entorno local. La aplicación compila y pasa el linter correctamente.

La app desplegada se veía pero el login con Google fallaba (el dominio de Render no estaba autorizado en Firebase). Se decidió eliminar la autenticación para simplificar el acceso.

En la aplicación desplegada, el inicio de sesión con Google falla y no permite entrar. Quita por completo el sistema de autenticación con Google/Firebase: la app debe abrir directamente en el dashboard (Panel) principal, sin pedir login ni iniciar sesión. Elimina la pantalla de inicio de sesión y cualquier verificación que bloquee el acceso. Mantén todo el resto de la aplicación igual (las pantallas, la base de datos, las importaciones y las funcionalidades). El objetivo es que la app sea accesible directamente, tanto en local como en el despliegue de Render.

Eliminé el sistema de autenticación tanto del backend (`middleware requireAuth`, creando un usuario por defecto persistente) como del frontend (inicializando el estado con datos predeterminados y eliminando los hooks de sesión de Firebase). La app ahora accede directamente al panel principal sin pantalla de login.

Tras conectar una base de datos PostgreSQL creada en Render, las importaciones no guardaban datos (error `ECONNREFUSED`). Se diagnosticó que el código no leía la variable `DATABASE_URL` y le faltaba SSL. Prompts utilizados:

Mi aplicación desplegada en Render no logra conectarse a la base de datos PostgreSQL. Los logs muestran el error `ECONNREFUSED` al intentar consultar la tabla con Drizzle ORM. Configuré una variable `DATABASE_URL` con la URL de conexión. Revisa el código del servidor y la configuración de Drizzle y dime qué nombre exacto de variable de entorno espera el código para la cadena de conexión, y si el código apunta a localhost por defecto. No cambies nada todavía, solo dame el diagnóstico.

Indicó que el código no esperaba una variable `DATABASE_URL` sino variables separadas (`SQL_HOST`, `SQL_USER`, `SQL_PASSWORD`, `SQL_DB_NAME`), por lo que al estar vacías el cliente asumía localhost y producía `ECONNREFUSED`. Además, faltaba la configuración SSL obligatoria para bases de datos gestionadas en la nube como Render.

Aplica la solución que propusiste para conectar la app a PostgreSQL en Render. Reconfigura `src/db/index.ts` para que el Pool de conexión use `connectionString: process.env.DATABASE_URL` y habilite SSL de forma condicional (`ssl: { rejectUnauthorized: false }` cuando exista `DATABASE_URL`, para entornos en la nube como Render). Haz también compatible `src/db/drizzle.config.ts` para que lea la cadena de conexión completa desde `DATABASE_URL` de forma prioritaria. El cambio debe mantener la compatibilidad para que la app siga funcionando también en el entorno local. Explicame qué cambiaste.

Modificó la función `createPool` para priorizar `DATABASE_URL`, añadió lógica de SSL condicional que se activa automáticamente para conexiones externas (no localhost), y preservó la compatibilidad local mediante las variables separadas como respaldo. Documentó la variable `DATABASE_URL` en el archivo `.env.example`.

Se explicó la diferencia entre la "Internal Database URL" (para la conexión entre el web service y la base de datos dentro de Render) y la "External Database URL" (para conexiones desde fuera, como Google AI Studio). Se configuró la URL interna en el web service de Render y la URL externa en AI Studio, de modo que ambos entornos compartan la misma base de datos PostgreSQL de Render. Se advirtió que al apuntar AI Studio a la base de Render (vacía), la app quedaría temporalmente sin datos hasta reimportar los archivos Excel.

¿Qué pasa si presiono el botón de empezar de cero en Render? ¿Debería quedarme todo vacío y yo después subir manual las cosas? / Acepto el riesgo de unificar ambos entornos en la misma base de datos.

Se aclaró que, al compartir la misma base de datos, vaciar desde Render también vaciaría AI Studio, y que para repoblar bastaba con reimportar los cuatro archivos Excel (clientes, proyectos, facturas y sesiones de tiempo) en el orden de dependencia correcto. Finalmente, tras hacer commit del cambio de código y redespargar, los datos aparecieron correctamente tanto en AI Studio como en la URL pública de Render.